

學士學位 論文

AXI 기반 CPU 설계 및 FPGA 구현

Design and FPGA Implementation of
a Custom CPU Based on AXI

指導教授 홍 유 표

東國大學校 電子電氣工學部

곽도영, 김기연, 남승현, 남상우

2021

學士學位 論文

AXI 기반 CPU 설계 및 FPGA 구현

곽도영, 김기연, 남승현, 남상우

指導教授 홍유표

이 論文을 學士學位 論文으로 提出함

2026년 6월 27일

곽도영, 김기연, 남승현, 남상우의 電子工學 學士學位 論文을 認准함

2026년 6월 27일

委員長_____ (印)

委 員_____ (印)

委 員_____ (印)

東國大學校 電子電氣工學部

목 차

제 I 장 서론	1
제 II 장 AXI protocol의 이해 및 동작 분석	2
2.1 AXI Handshake	2
2.2 Handshake 과정	2
2.3 AXI protocol의 동작	3
제 III 장 Custom CPU 설계 및 구현	5
3.1. CPU Core 설계	5
3.2. AXI interface 설계	7
제 IV 장 AXI 기반 Slave 컨트롤러 및 시스템 통합	9
4.1. AXI Memory Controller 설계	9
4.2. AXI UART Controller 설계	11
4.3 AXI Segment Controller 설계	13
제 V 장 FPGA 구현 및 검증 결과	15
5.1 RTL Simulation을 통한 검증	15
5.2 ILA 및 STA를 이용한 실시간 하드웨어 분석	18
5.3 외부 인터페이스를 통한 최종 시스템 통합 및 물리적 검증	20
제 VI 장 결론	22
참고문헌	23
ABSTRACT	24

제 I 장 서론

최근 FPGA를 이용한 디지털 시스템 설계에서는 CPU, 메모리, 주변 장치 간의 데이터 통신을 위해 표준 버스 구조가 사용되고 있다. 그중 AXI는 모듈 간 연결의 편리성으로 인해 FPGA 기반 시스템 설계에서 많이 활용되는 인터페이스 구조이다.

AXI 인터페이스를 사용하면 CPU와 메모리, UART와 같은 주변 장치를 하나의 통신 구조 안에서 연결할 수 있으며, 여러 장치를 효율적으로 제어할 수 있다는 장점이 있다.

하지만 학부 과정에서 진행하는 CPU 설계 실습은 대부분 시뮬레이션 환경에서 기능 동작을 확인하는 수준에서 끝나는 경우가 많으며, 베릴로그 코드로 설계한 CPU가 fpga보드에서도 올바르게 동작하도록 하기 위해서는 시뮬레이션상에서는 확인할 수 없었던 여러 문제들을 해결해야 한다. 예를들어 cpu와 메모리간의 데이터를 주고받는 타이밍 문제나 CPU의 12비트 주소를 32비트의 AXI주소로 변경해 줘야하는 등의 문제로 인해 예상과 다른 결과가 발생할 수 있다.

따라서 FPGA 환경에서 CPU, master, slave, ip bram, uart등을 AXI신호로 연결하여 정상 동작하는지를 검증하는 과정은 디지털 시스템 설계 측면에서 의미가 있다고 할 수 있다.

이번 프로젝트에서는 Verilog로 설계한 CPU를 FPGA 보드에 올려서 시뮬레이션 동작과 동일한 결과를 보여주는지 확인하였고, AXI신호를 통해 메모리 및 주변 장치와 통신하도록 시스템을 구성하였다. CPU의 신호를 AXI 신호로 변경하여 처리할 수 있도록 Master를 설계하였으며, 이를 통해 CPU가 하나의 장치뿐만 아니라 여러 장치와 통신할 수 있도록 하였다.

또한 FPGA 내부 메모리인 BRAM을 활용하여 데이터 저장 공간을 구성하였으며, COE 파일을 이용하여 초기 프로그램 및 데이터를 저장하였다. CPU는 AXI 인터페이스를 통해 BRAM에 데이터를 읽고 쓰는 동작을 수행하도록 설계하였다.

추가적으로 UART를 Slave 형태로 연결하여 CPU와 UART 간의 데이터 통신도 구현하였다. 이를 통해 AXI 구조의 장점 중 하나인 여러 장치와의 연결 및 확장 가능성을 확인할 수 있었다.

제 II 장 AXI protocol의 이해 및 동작 분석

2.1 AXI handshake

Handshake는 정보기술 및 디지털 통신 분야에서 데이터를 주고받기 전에 송신측과 수신측이 서로 데이터 전송이 가능한 상태인지 확인하는 과정이며, ‘사전준비, 협상과정’이라는 단어의 의미처럼 통신 준비 상태를 확인하는 것이며, 실제 데이터를 전달하기 전에 수행되어진다.

AXI 프로토콜에서는 각 채널마다 VALID와 READY 신호를 사용하여 Master와 Slave간의 Handshake를 수행한다. 데이터 전송은 VALID 와 READY신호가 동시에 HIGH인 경우에만 수행된다.

-VALID : 주소, 데이터 또는 제어 정보가 버스 상에 유효하게 준비되었음을 나타내는 신호이다.

-READY : 송신측의 데이터를 받아들일 공간이나 처리 준비가 완료되었음을 나타내는 신호이다.

2.2 Handshake 과정

2.2.1 VALID 신호가 앞서는 경우

그림 2.1은 송신 측이 데이터를 먼저 준비한 후 수신 측의 준비를 기다리는 상황이다. 송신측은 데이터와 함께 VALID 신호를 활성화하며, 수신 측의 READY 신호가 활성화될 때까지 현재 데이터를 유지한다. 이후 수신 측이 READY 신호를 활성화하면 두 신호가 동시에 활성화된 시점에서 데이터 전송이 완료된다.

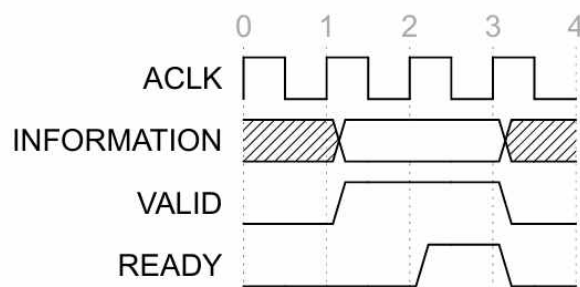


그림 2.1 VALID before READY handshake

2.2.2 READY 신호가 앞서는 경우

그림 2.2 는 수신 측이 먼저 데이터를 수신할 준비를 완료한 상태이다. 수신 측은 READY 신호를 활성화한 채 대기하며, 이후 송신 측이 데이터를 준비하고 VALID 신호를 활성화한다. 두 신호가 동시에 활성화되는 시점에서 데이터 전송이 이루어진다.

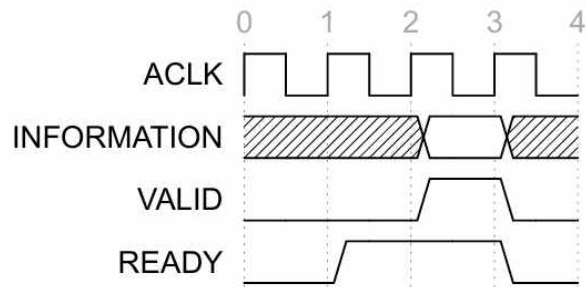


그림 2.2 READY before VALID handshake

2.2.3 VALID신호와 READY신호가 동시에 활성화된 경우

그림 2.3 송신 측과 수신 측이 동일한 시점에 각각 데이터 전송과 수신 준비를 완료한 상황이다. VALID 신호와 READY 신호가 동시에 활성화되므로 다음 클럭에서 바로 데이터 전송이 수행된다.

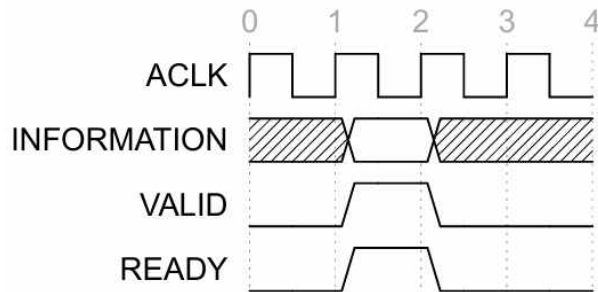


그림2.3 VALID with READY handshake

2.3. AXI protocol의 동작

2.3.1 AXI의 독립적인 5개의 채널

채널 이름 (Channel)	방향 (Direction)	역할
Read Address	Master -> Slave	읽기 대상 주소 전달
Read Data	Slave -> Master	읽어온 데이터 및 읽기 완료 응답 전송
Write Address	Master -> Slave	쓰기 대상 주소 전달
Write Data	Master -> Slave	실제 슬레이브에 저장할 데이터 전송
Write Response	Slave -> Master	쓰기 작업 완료 여부(응답 상태) 전달

Read동작을 수행하는 경우에 Read Address채널과 Read Data채널이 사용된다. Write동작하는 경우에 Write Address채널, Write Data채널, Write Response채널이 사용된다.

Read동작과 달리 Write동작에서는 Write Response채널이 필요하다. 왜냐하면 Read의 경우 Read Data채널을 통해서 ‘읽기완료응답’을 전송할 수 있지만, Write의 경우 Write Data채널의 반대방향으로 Write Response신호를 보내야 하므로, 별개의 Write response 채널이 필요하다.

2.3.2 AXI에서 READ/WRITE의 동작

1. Read동작

Read동작시에는 Read Address채널, Read Data채널이 사용된다.

Read Address채널을 통해서 Master가 Slave에게 데이터를 읽어올 주소를 보내고, Read Data채널을 통해서 Slave로 부터 데이터를 받아온다. 각각의 채널은 독립적으로 동작하기에, Master가 데이터를 읽어올 여러개의 주소를 보내는 와중에 Slave로 부터 read data를 받는 것이 가능하며, 다양한 경우의 조합으로 동작한다.

2. Write동작

Write동작시에는 Write Address채널, Write Data채널, Write Response채널이 사용된다.

Write Address 채널을 통해 Master가 Slave에게 데이터를 저장할 주소를 전송하고, Write Data 채널을 통해 데이터를 전달하며, 최종적으로 Write Response 채널을 통해 Slave로부터 쓰기 작업의 완료 응답을 받아온다.

Read 동작과 마찬가지로 각각의 채널은 독립적으로 동작하기 때문에, Master가 연속해서 Write Address를 보내거나, 데이터를 전송하는 중에 Slave의 Write response를 받는 것이 가능하며, 다양한 경우의 조합들로 유연하게 동작한다.

2.3.3 AXI채널의 분리해서 얻는 이점

1. Outstanding Transaction

AXI는 주소 채널과 데이터 채널이 독립적으로 동작한다.

기존의 AHB와 같은 버스 구조에서는 주소 전송 이후 해당 데이터의 처리가 완료될 때까지 다음 요청을 수행하기 어려웠다. 따라서 하나의 전송 과정이 끝나기 전까지 버스 자원이 대기 상태에 놓이게 되는 문제가 존재하였다.

반면 AXI는 읽기 주소 채널(AR)과 읽기 데이터 채널(R)이 분리되어 있으므로, 이전 데이터 응답이 완료되지 않은 상태에서도 새로운 주소 요청을 연속적으로 전달할 수 있다. 이를 통해 여러 개의 전송 요청을 동시에 유지할 수 있으며 결과적으로 버스 대기 시간이 감소하고 전체 데이터 처리량이 향상된다.

2. 읽기(Read)와 쓰기(Write)의 완전한 동시성

AXI는 Read채널과 Write채널을 완전히 독립적으로 구성하여 읽기와 쓰기 동작의 동시 수행이 가능하다. 기존 버스 구조에서는 읽기와 쓰기가 동일한 자원을 공유하기 때문에 한 시점에 하나의 동작만 수행할 수 있었다. 그러나 AXI는 읽기 전용 채널(AR, R)과 쓰기 전용 채널(AW, W, B)을 분리함으로써 읽기 데이터 전송과 쓰기 데이터 전송을 동시에 처리할 수 있다. 예를들어 마스터가 슬레이브로부터 데이터를 Read 중에 다른 데이터를 Write하기 위해서 주소와 데이터를 보내는 동작을 진행할 수 있다. 이러한 구조는 시스템 내부 데이터 전송의 병목 현상을 줄이고, 고속 데이터 처리가 요구되는 SoC환경에서 높은 성능을 제공하므로 버스 프로토콜로서 많이 활용되고 있다.

제III장 Custom CPU 설계 및 구현

3.1 CPU Core 설계

CPU Core는 최상위 모듈 axi_CPU 내 인스턴스되어 AXI 버스와 연동된다. 기존 설계한 mano instruction 기반 CPU에서 axi interface를 연동시키기 위해 몇 가지 수정이 가해졌다.

3.1.1 공통 버스 제거

전통적인 기본 CPU 아키텍처(Mano Machine)에서는 여러 레지스터(AR, PC, DR, AC, IR 등) 간의 데이터 이동을 위해 하나의 경로를 공유하는 '공통 버스(Common Bus)' 시스템을 사용한다.

하지만 이는 다중 데이터 전송을 제약하는 병목 요인으로 작용한다. 특히 본 시스템에 적용된 AXI4-Lite 통신 규격은 쓰기 트랜잭션 시 주소 채널(AW)과 데이터 채널(W)을 동시에 전송할 수 있는 병렬성을 지원하지만, 내부 공통 버스를 사용할 경우 주소와 데이터를 순차적으로 버스에 실어야 하므로 클럭 사이클의 낭비와 FSM 제어의 복잡성이 증가한다. 이를 해결하기 위해, 본 설계에서는 공통 버스 구조를 배제하고 레지스터 간 직접 연결(Point-to-Point) 구조를 구현하였다. 이를 통해 특정 상태(예: 메모리 쓰기)에서 mem_addr와 mem_wdata를 단일 사이클에 동시에 AXI 인터페이스로 출력할 수 있어 버스 프로토콜의 병렬 전송 효율을 극대화하였다.

3.1.2 handshake interface, stall logic 추가

cpu_core 모듈은 본래 독립적인 타이밍으로 동작하는 프로세서였으나, 외부 AXI4-Lite Master 인터페이스와 정합성을 맞추고 대기 상태를 제어하기 위해 내부에 핸드셰이킹 포트와 SC를 Stall 시키는 로직이 추가되었다.

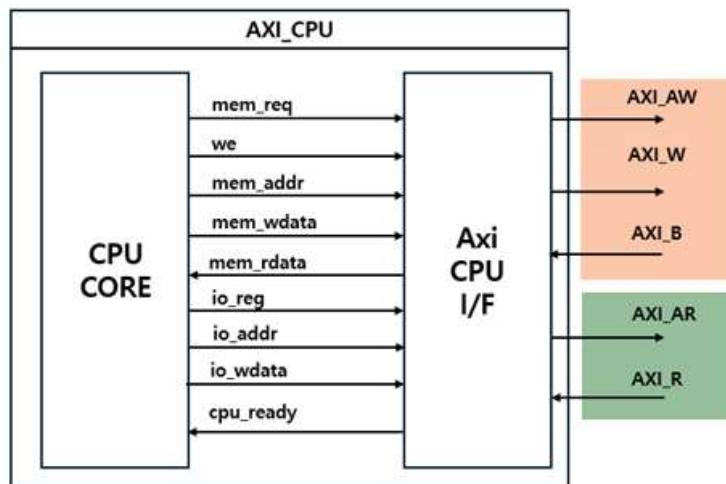


그림3.1 axi_cpu block desgin

1) 버스 핸드셰이킹을 위한 전용 인터페이스 포트 구현

AXI 인터페이스 모듈과 통신하기 위해 CPU 최상위 포트에 다음과 같은 제어 및 데이터 신호가 추가되었다.

-제어 출력 포트 (mem_req, we): 메모리 접근이 유효함을 알리는 요청 신호와 읽기/쓰기를 구분하는 신호가 추가되어 AXI 주소 채널의 valid 발생 시점을 제어한다.

-동기화 입력 포트 (cpu_ready): AXI 브리지가 외부 슬레이브(BRAM 또는 I/O)와의 트랜잭션을 끝내고 완료 응답을 보냈을 때 받아들이는 신호로, CPU 내부 상태 머신의 흐름을 결정한다.

-I/O 전용 포트 (io_req, io_addr, io_wdata): 일반 BRAM 메모리 영역 외에 UART 및 7-Segment와 같은 주변장치 액세스를 위한 별도의 I/O 채널 신호가 구성되었다.

2) 대기 상태(Wait State) 삽입을 위한 FSM(State Machine) 변형

기존 Mano 아키텍처 등은 각 클럭 사이클마다 시퀀스 카운터(SC)가 T0 -> T1 -> T2로 무조건 1씩 증가하는 구조를 가진다. 그러나 응답 지연이 존재하는 AXI 버스를 지원하기 위해, 버스 요청 후 응답을 기다리는 정지(Stall) 메커니즘이 FSM에 포함되었다.

-명령어 페치(Instruction Fetch) 단계 (T1):

T0에서 주소를 인가하고 mem_req를 올린 후, T1 상태에서 cpu_ready 신호가 들어왔는지 검사한다. cpu_ready가 1인 경우에만 버스에서 데이터를 읽어 명령어 레지스터(IR)에 저장하고 다음 상태(T2)로 전송하며, 만약 0이라면 SC <= SC; 를 수행하여 CPU가 현재 상태에서 대기하도록 제어한다.

-데이터 메모리 읽기/쓰기 단계 (T6):

AND, ADD, LDA 등의 메모리 참조 명령어 실행 시, T5 영역에서 mem_req를 활성화한 후 T6 상태에서 동일하게 cpu_ready 신호를 추적한다. 버스 읽기가 완료되는 시점에만 누산기(AC)나 데이터 레지스터(DR)에 값을 업데이트하고 시퀀스 카운터를 4'h0으로 리셋하여 다음 명령어로 전환한다.

3.1.3 I/O 명령어

본 설계는 CPU 내부의 누산기(AC) 데이터를 외부 I/O 장치(UART, 7-Segment 등)로 출력하기 위해 별도의 I/O 전용 포트를 두는 대신, AXI 버스 상의 특정 메모리 주소 영역에 I/O 장치를 매핑하는 Memory-Mapped I/O 방식을 채택하였다.

명령어 해독 및 I/O 요청 발생

CPU 코어의 상태 머신 T3 사이클(SC == 4'h3)에서 명령어를 디코딩하여 I/O 동작을 결정한다.

명령어 포맷 판별: 명령어의 Opcode를 해독한 결과가 7(D[7] == 1)이고, 최상위 비트인 간접 주소 지정 비트가 1(I == 1)일 때 이를 I/O 제어 명령어로 인식한다.

장치 식별 및 주소 할당: 명령어 레지스터의 하위 12비트(IR[11:0]) 값을 장치 식별 코드로 사용하여 타겟 장치의 절대 주소를 매핑한다.

12'h001: UART Tx Write 명령. AXI 타겟 주소를 32'h44A10000으로 설정한다.

12'h002: 7-Segment Write 명령. AXI 타겟 주소를 32'h44A00000으로 설정한다.

데이터 및 신호 전달: 전송할 데이터는 누산기(AC)의 값을 io_wdata로 출력하며, AXI 인터페이스 모듈에 I/O 트랜잭션 시작을 알리기 위해 io_req 신호를 활성화(High)한다.

3.2 axi interface 설계

Axi_cpu_interface는 16비트 아키텍처의 커스텀 CPU 코어와 32비트 AXI 버스 간의 통신을 중재한다. 이를 위해 AXI 프로토콜의 5개 독립 채널에 대한 신호들을 다음과 같이 매핑 및 제어하도록 구현하였다.

3.2.1 AXI BUS 신호

1) 공통 핸드셰이크 규격 및 동작 원리

AMBA AXI 프로토콜의 5개 독립 채널(AW, W, B, AR, R)은 valid와 ready 신호 쌍을 기반으로 하는 handshake 구조를 채택하고 있다.

본 설계의 모든 플립플롭은 시스템 클럭(axi_aclk)의 상승 에지(Posedge)에 동기화되어 동작하며, valid와 ready 신호가 동시에 High(1)로 활성화된 상태에서 클럭 상승 에지를 만나는 시점에 Transaction이 성공적으로 완료(Handshake)된다. 이 메커니즘은 송수신 장치 간의 동작 속도 차이로 인한 데이터 유실을 방지하며, 버스의 흐름 제어를 유연하게 수행하도록 돕는다.



그림3.2 읽기 주소 handshake

2) 쓰기 주소 채널 (Write Address Channel, AW)

axi_awaddr (Output, 32-bit): 쓰기 동작을 수행할 목표 슬레이브의 32비트 절대 주소를 출력한다. CPU가 요청한 메모리 주소 영역(32'h40000000 Base) 또는 I/O 영역(32'h44A00000 등)이 변환되어 이 포트로 전달된다.

axi_awvalid (Output): Master가 유효한 쓰기 주소와 제어 신호를 출력하고 있음을 나타내는 출력 신호이다.

axi_awready (Input): Slave가 주소를 수신할 준비가 되었음을 Master에게 알려주는 입력 신호이다.

2) 쓰기 데이터 채널 (Write Data Channel, W)

axi_wdata (Output, 32-bit): 외부 슬레이브에 기입할 32비트 버스 규격의 데이터이다. CPU 내부의 16비트 데이터(mem_wdata, io_wdata)는 상위 16비트에 0이 채워진 채 패딩(Padding)되어 이 포트로 출력된다.

axi_wvalid (Output): 유효한 쓰기 데이터 신호가 버스에 준비되었음을 알리는 신호이다. 주소 채널의 유효 신호(awvalid)와 동시에 활성화되어 병렬 전송 효율을 극대화한다.

axi_wready (Input): Slave가 쓰기 데이터를 받아들일 수 있는 상태임을 나타내는 입력 신호이다.

3) 쓰기 응답 채널 (Write Response Channel, B)

axi_bresp (Input, 2-bit): 슬레이브로부터 전송된 쓰기 작업의 성공 여부 응답 상태 코드이다. (2'b00: OKAY, 2'b10: SLVERR 등)

axi_bvalid (Input): Slave가 유효한 쓰기 응답 코드를 출력하고 있음을 알리는 신호이다. 이 신호가 인가되어야 Master의 쓰기 상태 플래그(write_in_progress)가 해제된다.

axi_bready (Output): Master가 쓰기 응답을 수신할 준비가 되었음을 응답 채널에 알리는 신호이다.

4) 읽기 주소 채널 (Read Address Channel, AR)

axi_araddr (Output, 32-bit): 읽기 동작을 수행할 목표 슬레이브의 32비트 절대 주소를 출력한다. CPU가 전달한 12비트 메모리 주소(mem_addr)를 바이트 단위 주소 매핑 규칙에 맞추어 연산한 최종 주소가 출력된다.

axi_arvalid (Output): 읽기 주소 플래그가 활성화되는 순간 Master가 유효한 주소를 출력하고 있음을 버스에 알리는 신호이다.

axi_arready (Input): Slave가 읽기 주소를 접수할 준비가 완료되었음을 나타내는 신호이다.

5) 읽기 데이터 채널 (Read Data Channel, R)

axi_rdata (Input, 32-bit): 주소가 지정된 슬레이브 영역으로부터 읽어온 32비트 병렬 데이터이다. 인터페이스 모듈은 이 데이터 중 하위 16비트([15:0])만을 발체하여 CPU 코어의 데이터 레지스터(DR, IR) 및 누산기(AC) 연산에 제공한다.

axi_rresp (Input, 2-bit): 읽기 작업에 대한 슬레이브 측의 상태 응답 코드이다.

axi_rvalid (Input): Slave가 요청된 버스 데이터를 안전하게 준비하여 출력하고 있음을 알리는 유효 신호이다.

axi_rready (Output): Master가 읽기 데이터를 수신할 준비가 되었음을 알리는 신호이며, 읽기 상태 플래그(read_in_progress)가 서 있는 동안 유지되다가 데이터가 포트에 도착(rvalid 적격)하면 데이터를 캡처한 후 낮아진다.

3.2.2 AXI BUS 주소

AXI 프로토콜은 메모리의 1바이트(8-bit)마다 주소를 1씩 할당하는 방식을 사용한다. 본 설계에서 AXI 데이터 버스는 slave와의 범용성을 위해 32비트(4바이트) 폭을 가진다. 따라서 하나의 데이터 워드가 바뀔 때마다 주소는 4씩 증가해야 정렬이 맞다. 그러나 CPU 코어 내부에서는 12비트 폭의 워드 단위 주소(mem_addr)를 사용한다. 이를 수정 없이 AXI 버스에 입력하면, AXI 슬레이브는 바이트 단위로 해석하므로 주소 격차가 발생하여 데이터를 정상적으로 읽고 쓸 수 없다. 따라서 주소 변환이 필요하다. 인터페이스 모듈은 CPU의 12비트 워드 주소를 하위 2비트 2'b00과 결합(Shift Left 2)하여 4바이트 정렬을 맞춘다. 이후 상위 18'b0 패딩으로 32비트 확장한 주소에 BRAM 베이스 주소를 더해 최종 물리 주소를 생성한다.

제Ⅳ장 AXI 기반 Slave 컨트롤러 및 시스템 통합

이번 프로젝트에서 CPU 코어와 연동하기 위해 최종 구현한 슬레이브 모듈은 주 기억장치인 BRAM과 외부 통신용 UART 컨트롤러, 그리고 출력용 Segment 컨트롤러로 구성된다. 이 중에서 UART 파트는 단순 데이터 수신을 넘어 마스터 제어 기능까지 포함하도록 직접 설계해 여타 모듈과 차별화된 하드웨어 구조를 갖췄다. 본 장은 이렇게 구성된 각 주변장치들이 CPU와 어떻게 데이터를 교환하는지 구체적인 동작 원리를 다룬다. 메모리 컨트롤러를 시작으로 UART 및 7-세그먼트 컨트롤러의 타이밍 제어 방식과 논리적 구현 과정을 순차적으로 정리했다.

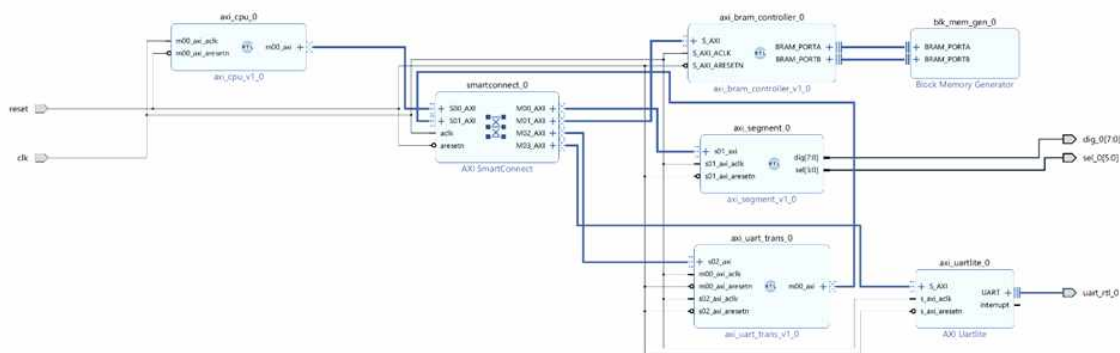


그림 4.1 전체 시스템 block desgin

4.1장 AXI Memory Controller 설계

1) Memory

시스템에 탑재된 메모리는 Xilinx Vivado의 Block Memory Generator IP 기반으로 데이터 폭 16비트, 깊이 4096 규격의 Simple Dual-port BRAM을 적용했다. 데이터 충돌을 피하고자 Port A(쓰기)와 Port B(읽기)의 역할을 분리하여 양방향 제어가 가능하도록 구성했으며, 4096의 깊이에 대응하기 위해 0x0000~0x0FFF 구간을 16비트 주소 버스로 맵핑했다. 인터페이스를 설계할 때 가장 유의했던 점은 동기식 메모리의 지연 시간이다. Datasheet대로 유효 주소 인가 시점으로부터 정확히 1 클럭 사이클을 대기해야만 포트로 올바른 데이터가 출력된다는 점을 제어 로직에 반영했다.

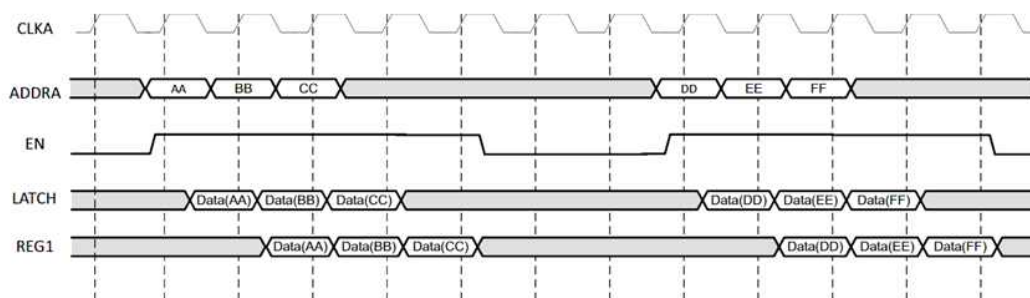


그림 4.2 ADDRA에 주소가 인가된 후 1clk 지연 후에 REG1에 Data(AA)가 나타나는 모습

2) Memory Controller 인터페이스 및 제어 구조

CPU와 BRAM 사이에서 핸드셰이크 기반의 안정적인 고속 통신을 중재하는 것이 Memory Controller의 역할이다. 프로토콜 변환 과정에서 주소 체계 불일치 문제를 해결하는 것이 가장 까다로웠다. 우선 32비트 AXI 주소 버스 신호를 16비트 메모리 규격에 그대로 연결하면 오작동이 발생하므로 상위 비트를 잘라내는 마스킹(Masking) 기법을 적용했다. 또한 바이트 단위로 주소가 4씩 증가하는 AXI 방식과 1씩 늘어나는 BRAM의 워드 단위 방식 간의 간극을 동기화하기 위해, 컨트롤러 설계 단계에서 별도의 주소 매핑 변환 로직을 회로 내부에 추가해 넣었다.

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
/axi_cpu_0					
/axi_cpu_0/m00_axi (32 address bits : 4G)					
/axi_bram_controller_0/S_AXI	S_AXI	reg0	0x4000_0000	4K	0x4000_0FFF
/axi_segment_0/s01_axi	s01_axi	reg0	0x44A0_0000	64K	0x44A0_FFFF
/axi_uart_trans_0/s02_axi	s02_axi	reg0	0x44A1_0000	64K	0x44A1_FFFF
/axi_uartlite_0/S_AXI	S_AXI	Reg	0x4060_0000	64K	0x4060_FFFF
/axi_uart_trans_0					
/axi_uart_trans_0/m00_axi (32 address bits : 4G)					
/axi_bram_controller_0/S_AXI	S_AXI	reg0	0x4000_0000	4K	0x4000_0FFF
/axi_segment_0/s01_axi	s01_axi	reg0	0x44A0_0000	64K	0x44A0_FFFF
/axi_uart_trans_0/s02_axi	s02_axi	reg0	0x44A1_0000	64K	0x44A1_FFFF
/axi_uartlite_0/S_AXI	S_AXI	Reg	0x4060_0000	64K	0x4060_FFFF

그림 4.3 AXI 인터페이스 내 각 Slave 모듈의 주소 할당(Address Mapping)

컨트롤러의 입출력 핀(Pin-out)을 살펴보면, 입력 쪽에는 시스템 클럭(clk), 비동기 리셋(resetn), AXI 입력 채널 신호들과 BRAM에서 빼낸 읽기 데이터 버스(mem_rdata)가 위치한다. 반대로 출력 쪽에는 AXI 출력 채널 신호들과 BRAM을 제어할 주소 버스(mem_addr), 쓰기 데이터 버스(mem_wdata)가 자리 잡고 있다. 제어를 담당하는 내부 핵심 신호로는 현재 읽고 쓰기가 진행 중인지 알려주는 bram_write_run과 bram_read_run, 채널별 버퍼에 데이터가 찼는지 판별하는 aw_has_data, w_has_data, ar_has_data, 그리고 BRAM의 1 클럭 읽기 지연을 맞추기 위해 도입한 enb와 enb_d1 레지스터가 존재한다.

bram_write_run은 쓰기 주소 채널(aw_has_data)과 쓰기 데이터 채널(w_has_data) 신호가 동시에 들어와 유효해질 때 1로 셋(Set)되며, 현재 BRAM에 데이터를 쓰는 트랜잭션이 진행 중임을 시스템에 알린다. 반대로 bram_read_run은 읽기 주소 채널(ar_has_data)이 1로 들어왔지만 아직 AXI 읽기 유효 신호(axi_rvalid)가 0으로 대기 중일 때 1로 켜지면서 읽기 사이클이 시작되었음을 의미한다. Memory Controller 내부 코드는 AXI 프로토콜 특유의 채널 독립성을 최대한 살리고자 5개의 채널을 각각의 always 구문으로 철저히 분리해서 설계했다. 그리고 최종단에는 Port A와 Port B 신호를 하나로 모아 통제하는 두 개의 컨트롤 로직을 붙여 메모리를 유기적으로 제어하도록 구성했다.

3) 쓰기 트랜잭션 제어 메커니즘 (Write Transaction)

주소 채널과 데이터 채널이 비동기적으로 작동하는 AXI 규격 특성 탓에 트랜잭션 시점 제어는 필수적이다. 이를 위해 주소 및 데이터가 버퍼에 완전히 도착했는지 판별하는 bram_write_run 상태 플래그를 도입했다. 이 플래그가 1로 전환되어야만 BRAM Port A의 인에이블 신호(ena, wea)가 활성화되며 주소(mem_addr)와 데이터(mem_wdata) 매핑이 시

작된다. 반대로 둘 중 하나라도 지연 수신되어 플래그 값이 0에 머문다면, 인에이블 핀을 강제로 닫아버려 불완전한 상태로 메모리가 덮어쓰워지는 사고를 사전에 차단하도록 구성했다. 이후 응답 채널(B)은 플래그가 활성화된 시점을 타고 axi_bvalid 신호를 발생시킨다. 마스터 쪽의 bready와 결합해 핸드셰이크가 최종적으로 성립되면 유효 상태를 다시 0으로 초기화하여 쓰기 사이클을 매듭짓는다.

4) 읽기 트랜잭션 제어 메커니즘 (Read Transaction)

읽기 과정도 전체적인 핸드셰이크 흐름은 쓰기 동작과 궤를 같이하지만, 동기식 BRAM의 구조 때문에 데이터를 끄집어내려면 필연적으로 1 클럭 사이클의 지연(Read Latency)을 견뎌야 한다. 우리는 이 타이밍 공백을 동기화하기 위해 enb_d1이라는 시프트 레지스터를 설계에 끌어들었다. 컨트롤러는 매 클럭의 상승 에지(posedge clk)가 될 때마다 현재의 BRAM 메모리 활성화 신호(enb) 값을 enb_d1으로 한 칸씩 밀어내도록(Shift) 코딩되어 있다. CPU가 넘긴 읽기 주소를 정상적으로 접수했고 아직 데이터 출력이 끝나지 않아 bram_read_run이 1로 유지되는 동안, Port B 제어 로직은 enb 핀을 1로 활성화하고 BRAM 주소 버스에 타겟 주소를 밀어 넣는다. 주소가 들어가고 딱 1 클럭 지연이 지나면 BRAM 내부 파이프라인을 거친 유효 데이터가 mem_rdata에 모습을 드러내는데, 바로 이 시점에 한 박자 늦춰둔 enb_d1 레지스터의 값이 1로 올라오게 된다. 컨트롤러는 enb_d1이 1이 된 순간을 데이터가 준비된 시점으로 간주하여 AXI 읽기 유효 신호(axi_rvalid)를 1로 구동하고, mem_rdata로 튀어나온 값을 AXI 읽기 데이터 버스(axi_rdata)에 태워 CPU로 무사히 전송한다.

4.2장 AXI UART Controller 설계

1) UART

본 프로젝트에서 도입한 UART 모듈은 Xilinx Vivado에서 지원하는 uart_axilite IP를 가져와 썼다. UART를 도입한 목적은 크게 두 가지로 요약된다. 첫 번째는 향후 다중 Master-Slave 구조로 시스템을 키워갈 때, 각 AXI 블록이 제대로 통신하고 있는지 최종 연산 결과를 PC 시리얼 모니터로 띄워 교차 검증하기 위해서였다. 두 번째는 UART Controller를 우리 손으로 직접 설계해 보면서 그 내부 동작 원리를 하드웨어 관점에서 깊이 파고들어 보려는 의도였다. 이번 설계에서는 UART의 송신(Tx) 기능만을 활용했기 때문에 데이터 전송 파이프라인에 초점을 맞춰 서술한다.

UART(Universal Asynchronous Receiver/Transmitter)는 하드웨어 간의 직렬 통신을 위한 표준 프로토콜이다. 서로 클럭을 공유하지 않는 비동기 방식이라 사전에 약속한 보드레이트(Baud rate)에 의존해 데이터를 주고받는데, 우리 시스템은 9600 bps 속도에 맞추어 세팅했다. 병렬로 된 데이터를 직렬로 변환해 내보내며, 패킷의 뼈대는 Start Bit, Data Bits, Parity Bit, Stop Bit로 짜여 있다. 신호 라인이 1에서 0으로 떨어지는 하강 에지(Falling Edge)인 Start Bit를 신호탄 삼아 8비트 데이터를 순차적으로 쏘아 보내고, 다 보내고 나면 Stop Bit를 1로 띄워 전송을 완료한다. 구현을 단순화하기 위해 에러를 체크하는 Parity Bit는 본 설계에서 제외했다.

2) UART Controller

직접 하드웨어 코드로 짠 UART Controller는 내부적으로 세 가지 주요 블록으로 구성된다. 첫째는 CPU가 내리는 명령을 수신하는 AXI Slave 블록, 둘째는 넘겨받은 데이터를 알맞게 변환하는 내부 Transform 블록, 셋째는 가공된 데이터를 실제 UART IP로 쏘주는 AXI Master 블록이다.

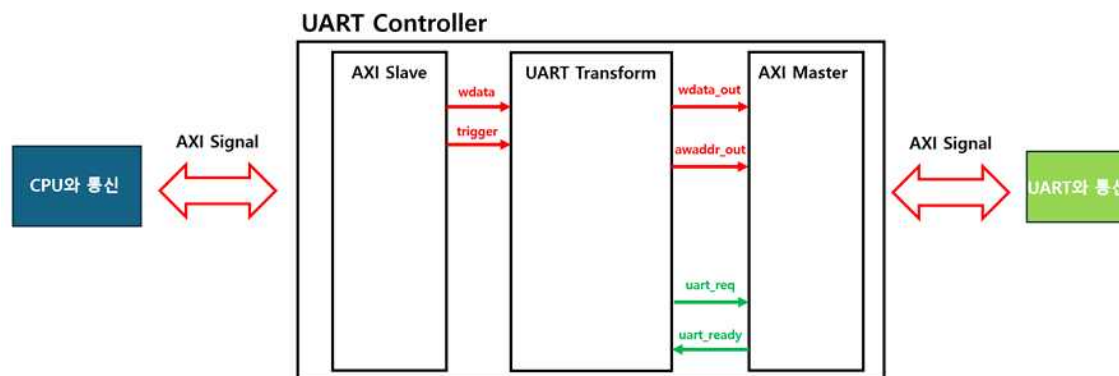


그림 4.4 UART Controller 블록 다이어그램(Block Diagram)

AXI Slave 블록은 앞서 다룬 Memory Controller와 비슷하게 각 채널별 always 구문을 통해 독립적으로 제어된다. 다만 UART는 오직 송신 전용으로 동작해서 읽기(Read) 기능이 불필요했기에, R 채널과 AR 채널 신호들에 더미(Dummy) 값을 고정으로 할당하여 통신 오류를 미연에 방지했다. 이 컨트롤러의 심장부라 할 수 있는 Transform 블록은 AXI로 수신한 16비트 데이터를 ASCII 문자로 바꾼 뒤, 1바이트씩 쪼개어 AXI Master를 통해 순차적으로 전송하는 역할을 수행한다. 이를 위해 hex2ascii라는 변환 함수를 구현했다. 4비트 단위로 끊어 읽은 값이 0x9 이하면 0x30을 더해 '0'~'9' 문자로 변환하고, 0x9를 넘어가면 0x37을 더해 'A'~'F' 문자로 변환해 주는 로직이다. Transform 블록의 제어 흐름은 유한 상태 머신(FSM)으로 묶어 U_IDLE, U_AW, U_W, U_NEXT 4가지 상태로 정의된다. 데이터를 쓰기 위해 폭이 8비트고 6개 요소(8x6)를 갖는 tx_buffer 2차원 배열을 핵심 레지스터로 활용했다.

-U_IDLE 상태: CPU가 타겟 주소에 데이터를 쓰는 즉시 16비트 쓰기 데이터(wdata)를 4비트씩 4조각으로 내어 tx_buffer[0]부터 [3]까지 각각 채워 넣는다. 나머지 tx_buffer[4]와 [5]에는 줄바꿈 동작을 구현하기 위해 Carriage Return('\r', 0x0D)과 Line Feed('\n', 0x0A) 문자를 끼워 넣었다. 만약 CPU로부터 쓰기 요청이 들어오지 않으면 계속 이 상태를 유지한다.

-U_AW 상태: req 신호를 1로 켜면서 미리 할당한 UART 타겟 주소(UART_TX_ADDR)를 AXI Master 쪽으로 전달한다. 동시에 wdata_out의 상위 24비트는 0으로 덮어버리고 (Padding) 하위 8비트에 현재 차례의 tx_buffer[tx_index] 값을 매핑해 출력한다. 리셋 시 tx_index가 0으로 초기화되므로 첫 사이클에선 tx_buffer[0]의 데이터가 할당된다.

-U_W 상태: 통신 준비가 끝났다는 ready 신호가 1로 올라오면 다음 상태인 U_NEXT로 넘어간다.

-U_NEXT 상태: tx_index 값이 5에 도달했는지 검사한다. 5라면 버퍼 전송을 모두 마친 셈이므로 U_IDLE 상태로 복귀한다. 아직 5가 아니라면 tx_index를 1 올리고 다시 U_AW 상

태로 돌아가 다음 문자를 보낼 준비를 한다. 이 과정을 돌며 tx_buffer[0]부터 [5]까지 데이터가 순차적으로 반복 전송된다.

마지막으로 AXI Master 블록은 신호가 마스터에서 슬레이브로 향한다는 방향성만 다를 뿐, 기본 구조는 AXI Slave 블록과 동일하다. Transform 블록이 보내온 req 신호로 통신 물꼬를 트고 ready를 돌려주는 메커니즘도 CPU 쪽 인터페이스 측과 똑같은 논리로 작동하게끔 맞췄다.

4.3장 AXI Segment Controller 설계

1) Segment

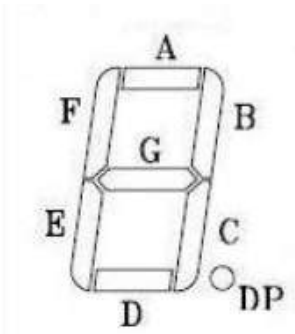


그림 4.5 Segment 기본 구조

그림 4.6 AX7035 Board에 내장되어 있는 Digital Tube

세그먼트(Segment)는 디지털 숫자를 표시하기 위한 LED 소자로서, 숫자의 형태를 띠는 7개의 막대형 LED(A부터 G까지)와 소수점을 찍는 원형 LED(DP) 하나로 이루어져 있다(그림 5). 이 세그먼트가 여러 개 이어진 구조를 Digital Tube라고 부르는데, 본 프로젝트에서 쓴 AX7035 FPGA 보드에는 그림 6처럼 생긴 Digital Tube가 박혀 있다. CPU가 연산해 낸 최종 결과값을 이 Digital Tube에 10진수 숫자로 띄우는 것이 Segment Controller의 궁극적인 목적이다.

2) Segment Controller

Segment Controller는 크게 CPU 쪽에서 전송하는 AXI 프로토콜 신호를 받아내는 AXI Slave 블록, 그리고 실제 7-세그먼트(7-Segment) LED 디스플레이를 깜빡이게 만드는 Segment Control 블록의 두 가지 모듈로 짜여 있다. AXI Slave 블록은 앞서 다뤘던 Memory Controller 및 UART Controller와 동일하게 각 채널별 always 구문을 적용하여 독립적인 제어를 수행했다. 7-세그먼트는 데이터를 수신(Write)하기만 하면 되는 소자라 읽기(Read) 기능은 불필요했다. 그래서 R 채널과 AR 채널에 관련된 신호들에는 더미(Dummy) 값을 강제로 할당하여 버스 통신상의 오류를 원천적으로 막아놓았다.

동적 스캐닝(Dynamic Scanning) 제어부

Segment Control 블록은 제일 끝단에서 세그먼트 LED를 구동하기 위해 8비트짜리 데이터 신호(dig)와 6비트짜리 자릿수 선택 신호(sel)를 직접 인가한다. AX7035 개발 보드의 데이터 시트를 보면, 해당 보드에는 6자리의 7-세그먼트 모듈이 내장되어 있다. 각 세그먼트마다 A~G, 그리고 DP까지 8개의 핀이 필요하므로, 이것 전부 독립적으로 제어하려면 물리적으로 총 48개(6 x 8)의 핀을 쏟아부어야 한다. 이는 하드웨어 자원 낭비가 심하고 배선 복잡도만 키우기 때문에, 8개의 데이터 핀을 모든 세그먼트가 돌려쓰게 묶어두고 6비트의 sel 신호로 활성화할 자릿수를 선택하는 구조로 구성되어 있다. 다만 이 핀 공유 방식을 쓰면 물리적으로

로는 한 번에 딱 한 자리만 제어할 수 있다. 우리는 이 한계를 극복하기 위해 동적 스캐닝 (Dynamic Scanning) 기법을 적용하여 6자리의 숫자가 동시에 점등되어 있는 것과 같은 시각적 효과를 구현했다. 동적 스캐닝은 인간의 눈이 인지하지 못할 만큼 짧은 시간 간격으로 각 자릿수를 번갈아 가며 켜다 켜서, 시각적 잔상 효과(Persistence of Vision)를 유도해 내는 기법이다. 이를 하드웨어로 구현하기 위해 50MHz짜리 시스템 클럭을 분주하여 딱 1ms 주기로 tick 신호를 발생시키는 타이머(Timer) 로직을 설계했다. 이 tick 신호가 1로 활성화 될 때마다 sel 신호를 다음 자리로 갱신(0부터 5까지 순환)해 주었다. 결과적으로 1ms마다 출력 위치가 스위칭되면서 6개의 세그먼트가 모두 흔들림 없이 켜져 있는 듯한 동적 디스플레이를 달성할 수 있었다.

3) Binary to BCD 변환기 구현 (Double Dabble 알고리즘)

Segment Control 블록 내부에는 CPU의 결과값을 디스플레이용으로 둔갑시키는 아주 중요한 데이터 변환 로직이 포함된다. CPU가 수신한 데이터는 16비트 형식의 2진수라서, 7-세그먼트 5자리에 나눠서 띄우려면 최대 5자리의 10진수로 변환해야만 했다. 우리는 이를 위해 Double Dabble (Shift-and-Add-3) 알고리즘을 끌어와 하드웨어 Binary to BCD 변환기로 직접 구현해 냈으며, 세부적인 동작 시퀀스는 다음과 같이 설계했다.

-레지스터 초기화: 총 36비트 크기의 시프트 레지스터(Shift Register)를 선언하고, 위쪽 상위 20비트는 0으로 썩 초기화한 뒤 하위 16비트에 입력받은 2진수 원본 데이터를 적재하여 세팅을 끝낸다.

-자릿수 분할 및 보정 (Add-3): 10진수로 변환될 상위 20비트 구간을 4비트씩 뚝뚝 끊어서 5개의 덩어리([35:32], [31:28], [27:24], [23:20], [19:16])로 분할하고, 각각을 BCD의 한 자릿수로 정의한다. 비트를 통째로 밀어내는 매 시프트 동작 직전에 이 4비트 구간들의 값을 하나하나 검사한다. 만약 5 이상인 값이 들어있다면 강제로 3을 더해주는(Add 3) 보정 작업을 거치는데, 이는 16진수가 넘어갈 타이밍을 10진수 체계의 자리올림에 알맞게 맞추기 위함이다. 반면 5 미만일 경우에는 연산을 수행하지 않고 그대로 둔다.

-시프트 연산 (Shift Left): Add-3 보정 단계가 무사히 끝나면 지체 없이 레지스터 전체 비트를 왼쪽으로 딱 1칸 시프트(Shift Left)한다.

-결과 도출: 2진수 데이터 폭과 동일하게 위 과정을 총 16회 반복 수행하면, 최종적으로 시프트 레지스터의 상위 20비트([35:16]) 구간에 완벽하게 10진수로 변환된 BCD 값이 도출되어 있다.

최종적으로 이렇게 뽑아낸 10진수 데이터는 각 자릿수에 맞게 분할되어 4비트 레지스터인 digit_val에 임시 저장된다. 그리고 1ms 주기로 튕겨주는 tick 신호에 동기화되어, 현재 활성화되어 있는 sel 자리에 알맞은 dig 신호로 최종 디코딩되어 출력된다.

제 V 장 FPGA 구현 및 검증 결과

5.0 CPU 동작 검증을 위한 성적 처리 프로그램

지금까지 만든 전체적인 시스템을 점검해보기 위해 CPU가 처리할 가상의 프로그램을 제작하였다. 100번지부터 109번지까지 저장되어 있는 임의의 성적값을 모두 합산하여 최종 결과값을 99번지에 저장하고 Segment에 10진수값을 띄운 뒤, UART를 통해 PC의 Serial Monitor에 16진수값을 띄우는 프로그램이다. 이를 위해, bram의 initial value값을 넣기 위해 coe 파일을 제작하였다(그림 5.1). 0번지부터 14번지까지는 우리가 수행할 16진수 instruction set을 순서대로 넣어놓았고, 99번지까지는 0을 넣어놓은 뒤, 100번지부터 109번지까지는 합산할 임의의 성적값 10개의 값을 16진수로 넣어놓았다. 성적의 총 합산값은 16진수로 031E, 10진수로 798이다. 우리가 만든 시스템이 정상동작했다면 segment 디스플레이에는 798이, serial monitor에는 031E가 표시되어야 한다.

```
memory_initialization_radix=16;
memory_initialization_vector=
710a, 7200, 7020, 3062, 7164, 3063, 7800, 9063, 6063, 6062, 4007, 3063, F002, F001,
400E, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0,
4d, 64, 37, 42, 4d, 58, 63, 4f, 43, 5a;
```

그림 5.1 bram에 들어가는 coe파일

명령어의 Flowchart는 다음과 같다. 카운터와 포인터를 설정해둔 뒤, 하나의 성적을 더하는 작업을 loop로 설정하고, 카운터값만큼 loop를 실행한다. 한번의 loop를 실행하고 나면, 포인터값에 1을 더해 다음 주소의 성적과 합산하는 알고리즘을 만들었다. 아래는 우리가 넣은 15개 명령어를 설명해놓은 표이다(표 5.2).

주소(번지)	명령어(16진수)		설명	RTL
0	710a	LDC	AC에 10(카운터) 저장	AC <- 10
1	7200	CMA	AC 값 보수화	AC <- ~AC
2	7020	INC	AC 값 + 1	AC <- AC+1
3	3062	STA	98번지에 카운터 저장	M[62] <- AC
4	7164	LDC	AC에 100(포인터) 저장	AC <- 100
5	3063	STA	99번지에 포인터 저장	M[63] <- AC
6	7800	CLA	AC 초기화	AC <- 0
7	9063	ADD(indirect)	AC + [포인터]번지의 값	AR <- M[63], AC <- AC+M[AR]
8	6063	ISZ	포인터 + 1 포인터=0이면 skip	M[63] <- M[63]+1, M[63]=0이면 PC <- PC+1
9	6062	ISZ	카운터 + 1 카운터=0이면 skip	M[62] <- M[62]+1, M[62]=0이면

				PC<-PC+1
10	4007	BUN	7번지로 Branch	PC <- 7
11	3063	STA	99번지에 결과값 저장	M[63] <- AC
12	F002	I/O(Segment)	Segment로 결과값 전송	-
13	F001	I/O(UART)	UART로 결과값 전송	-
14	400E	BUN	오작동 방지용 Branch	PC <- 14

표 5.2 성적처리 프로그램의 Instruction set 및 설명

5.1 RTL Simulation을 통한 검증

본 절에서는 설계된 AXI 기반 Custom CPU의 논리적 무결성을 증명하기 위해, 실제 FPGA 하드웨어 구현 전 소프트웨어 환경에서 진행한 RTL 시뮬레이션 결과를 분석한다. 회로의 설계가 의도와 일치하는지 확인하기 위해, CPU 코어 내부의 명령어 처리 사이클과 AXI 버스 트랜잭션 동작을 단계별로 검증하였다.

5.1.1 CPU Core 명령어 처리 및 접근 제어 검증

Custom CPU는 4비트 Sequence Counter(SC)를 기반으로 클럭 사이클에 맞춰 Fetch, Decode, Execute 단계를 수행한다. 시스템 통합 검증을 위해 CPU의 연산을 '메모리 접근 연산(Memory Access)'과 '입출력 주변장치 제어(I/O Access)'로 엄격히 구분하여 파형을 분석하였다.

(1) 메모리 접근 연산(Memory Access) 및 정규 Fetch 사이클 검증

본 연구에서 설계한 Custom CPU는 신호의 동기화 안정성과 하드웨어 타이밍 마진을 보장하기 위해 명령어 인출(Fetch) 단계를 3개의 독립된 클럭 사이클(SC=0, 1, 2)로 세분화하여 설계하였다. 일반적인 데이터 저장 과정의 신뢰성을 확인하기 위해 AC의 값을 메모리에 저장하는 STA 명령어를 중심으로 시뮬레이션을 수행하였다.

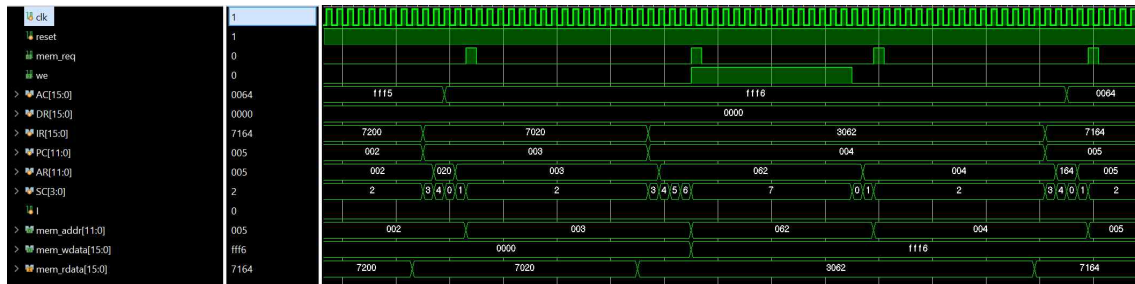


그림 5.3 CPU의 3단계 명령어 Fetch 및 메모리 접근 연산(STA) 시뮬레이션 파형

그림 5.3의 시뮬레이션 파형에서 볼 수 있듯이, CPU는 SC=0 (T0) 상태일 때 Program Counter(PC)의 값을 Address Register(AR)로 안정적으로 전송한다. 곧바로 이어지는 SC=1 (T1) 사이클에서는 동기화가 완료된 AR 값을 메모리 주소 버스(mem_addr)로 삽입함과 동시에 메모리 요청 신호(mem_req)를 High(1)로 만든다.

SC=2 (T2) 사이클에 진입하면 AXI 버스 인터페이스로부터 cpu_ready 응답이 반환될 때까지 대기 상태(Wait-state)를 유지하다가, 신호가 인가되는 즉시 Instruction Register(IR)에 명령어(0x3062)를 래치(Latch)하고 PC를 1 증가시키며 Fetch 동작을 완료한다.

명령어 인출이 완료된 후 SC=3 사이클에서는 Opcode(op)가 3(STA)으로 정상 해독(Decode)된다. 최종적으로 데이터 저장이 이루어지는 Execute 사이클(SC=6)을 살펴보면, 메

모리 접근 요청(mem_req)과 쓰기 활성화 신호(we)가 동시에 High(1)로 발생하며, 목적지 주소로 CPU 연산 결과인 0xFFFF6이 정상 기록됨을 확인하여 전체 파이프라인 구조의 무결성을 검증하였다.

(2) 입출력 주변장치 제어(I/O Access) 검증

이어서 CPU가 7-Segment와 UART 같은 외부 주변장치를 제어할 때 내보내는 I/O 분기 명령이 제때 시스템 버스(AXI)로 넘어가는지 살펴봤다. 우리가 구성한 I/O 명령어는 간접 주소지정 방식 플래그(I=1)와 특정 Opcode를 엮어서 움직이는 구조를 갖는다. 그림 5.4 시뮬레이션 파형을 보면 7-Segment 명령(0xF002)과 UART 명령(0xF001)이 연달아 실행되는 과정을 확인할 수 있다. 3단계의 정규 Fetch로 IR에 명령어를 가져온 다음, SC=3 사이클에서 op=7과 I=1로 알맞게 디코딩되는 모습이 관찰된다. 여기서 핵심은 Execute 사이클인 SC=4 시점이다. 일반적인 메모리 연산 때와 다르게 io_req 신호가 High(1)로 뜨면서, 첫 명령어(0xF002)는 7-Segment 주소(0x44A00000)로, 두 번째(0xF001)는 UART 주소(0x44A10000)로 각각 딜레이 없이 바뀌어 출력된다. 이때 누산기에 담겨있던 데이터(0x031E) 역시 io_wdata를 통해 고스란히 전달되는 것을 통해, CPU 코어가 여러 주변장치를 무리 없이 제어할 수 있는 전제조건을 확실히 갖추었다는 결론을 얻었다.

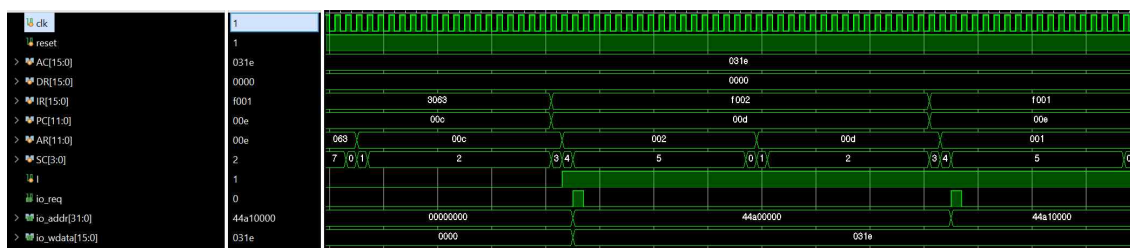


그림 5.4 간접 주소 지정 기반 I/O 명령어(7-Segment, UART) 연속 실행 파형

5.1.2 AXI 버스 트랜잭션 핸드셰이크 검증

앞서 살핀 CPU의 mem_req, io_req 신호는 AXI 인터페이스를 거쳐 AXI4-Lite 규격으로 바뀐다. 시뮬레이션에서는 Xilinx SMC IP를 버스로, BRAM 컨트롤러를 슬레이브로 두고 통합 테스트를 진행했다.

(1) AXI Write Transaction (쓰기 동작) 및 하드웨어 제어 검증

CPU가 데이터를 쓸 때 버스 핸드셰이크와 BRAM 제어 신호가 맞게 들어가는지 파형(그림 5.5)으로 확인했다. mem_req와 we 신호가 뜨면, 마스터 인터페이스는 지연을 줄이고자 주소(AWVALID)와 데이터(WVALID) 유효 신호를 같이 올린다. SMC IP와 슬레이브 컨트롤러는 AWREADY를 평소 High로 두기 때문에, AWVALID가 올라간 바로 그 클럭에서 핸드셰이크가 일어나 불필요한 대기시간이 사라진다. 이후 데이터가 슬레이브로 넘어가면 bram_write_run 플래그가 켜지고, BRAM 구동 신호(ena, wea)가 펄스 형태로 발생해 타겟 주소(062)에 데이터(FFF6)가 제대로 기록된다.

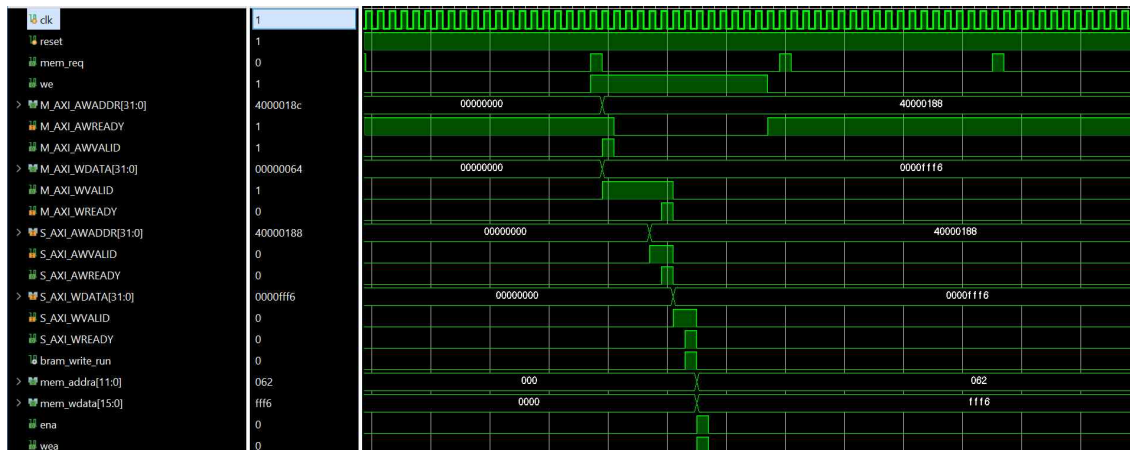


그림 5.5 AXI 쓰기(Write) 트랜잭션 및 BRAM 제어 신호 시뮬레이션 파형

(2) AXI Read Transaction (읽기 동작) 및 메모리 latency 제어 검증

데이터 읽기(그림 5.6)는 쓰기와 달리 주소를 보낸 뒤 BRAM에서 데이터가 나올 때까지 대기(Latency)해야 한다. we=0인 상태로 mem_req가 발생하면 마스터가 ARVALID를 띄워 읽기를 요청하고, 슬레이브는 ARREADY가 켜져 있어 1클럭 내에 주소를 받아간다. 주소를 받으면 슬레이브 측에서 bram_read_run을 띄워 BRAM의 enb를 켜다. 이때 BRAM의 1클럭 출력 지연에 맞추기 위해 내부 시프트 레지스터(enb_d1)를 거쳐 정확한 타이밍에 S_AXI_RVALID를 올린다. 반환된 데이터가 마스터를 지나 CPU로 들어가면 대기 상태(cpu_ack)가 풀리면서 읽기 동작이 문제없이 마무리된다.

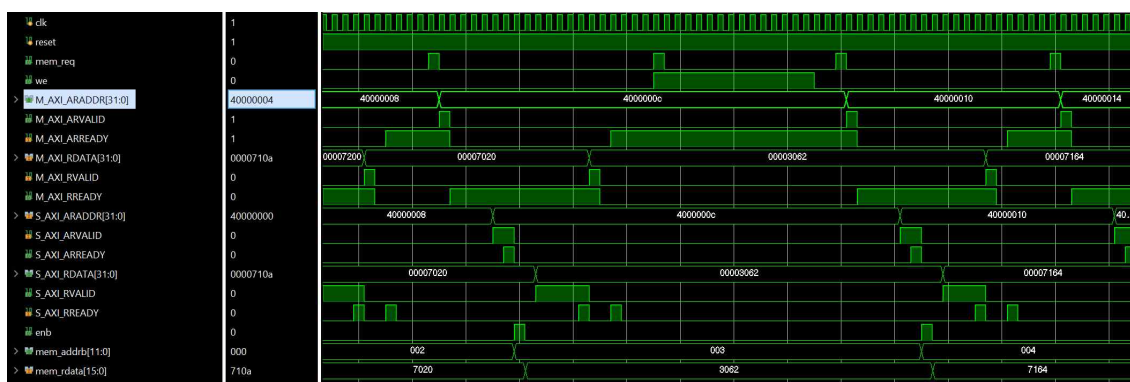


그림 5.6 AXI 읽기(Read) 트랜잭션 및 BRAM 레이턴시 동기화 시뮬레이션 파형

5.2 ILA 및 STA를 이용한 실시간 하드웨어 분석

시뮬레이션으로 확인한 로직을 바탕으로, 실제 FPGA 보드에 올렸을 때 물리적 타이밍에 문제가 없는지 STA(정적 타이밍 분석)와 ILA로 점검했다.

5.2.1 STA를 통한 물리적 타이밍 마진 검증

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 2.697 ns		Worst Hold Slack (WHS): 0.091 ns		Worst Pulse Width Slack (WPWS): 3.870 ns	
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): 0.000 ns		Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 0		Number of Failing Endpoints: 0	
Total Number of Endpoints: 1675		Total Number of Endpoints: 1675		Total Number of Endpoints: 1031	
All user specified timing constraints are met.					

그림 5.7 STA 결과 timing summary

보드 실측 전, Vivado STA로 시스템 지연 특성을 파악했다. 처음 50MHz(20ns) 클럭으로 합성했을 때 Setup 마진(WNS)이 12.341ns로 매우 넉넉했다. 클럭 주기에 비해 데이터 지연이 짧아 하드웨어 자원이 남는 상황이라, 최대 성능을 보려고 클럭을 100MHz(10ns)로 올려 다시 테스트했다. 10ns로 재배선한 결과 WNS가 2.697ns로 안정적인 양수를 유지하며 고속에서도 타이밍 클로저를 만족했다. 타이밍 병목을 유발하는 크리티컬 패스를 찾아보니 대부분 SMC IP 내부의 조합 회로망에 몰려 있었다. CPU나 인터페이스 내부의 지연은 셀 자체보다 소자 간 배선 지연(Net Delay)의 비중이 컸다. 결국 우리가 작성한 Verilog 코드를 다듬어 게이트를 줄이더라도 버스 인프라와 배선 지연 때문에 동작 주파수(Fmax)를 확 높이기 어렵다는 결론을 얻었다. 즉, 현재 아키텍처가 주어진 자원 내에서 최선의 타이밍 정합성을 갖췄다고 볼 수 있다.

5.2.2 하드웨어 검증 전략 및 ILA 도입 배경

칩 내부를 지나는 전기 신호는 시뮬레이션과 다르게 물리적 지연을 겪는다. 이를 확인하고자 동작 속도 그대로 하드웨어 신호를 찍어보는 ILA를 활용했다. 모든 레지스터를 다 보면 자원 낭비가 심하므로, CPU와 BRAM이 만나는 AXI 버스 경계에만 모니터링을 집중했다. 여기서 타이밍과 데이터가 맞게 뜬다면 내부 CPU 코어도 물리적 환경에서 정상 동작한다는 뜻이기 때문이다.

5.2.3 BRAM 읽기/쓰기 중심의 AXI 핸드셰이크 실시간 검증

STA 결과를 믿고 BRAM IP 연동 과정을 ILA로 캡처했다. 쓰기(AWVALID=1)와 읽기(ARVALID=1)를 트리거로 잡았다..

(1) AXI 쓰기(Write) 동작 실측

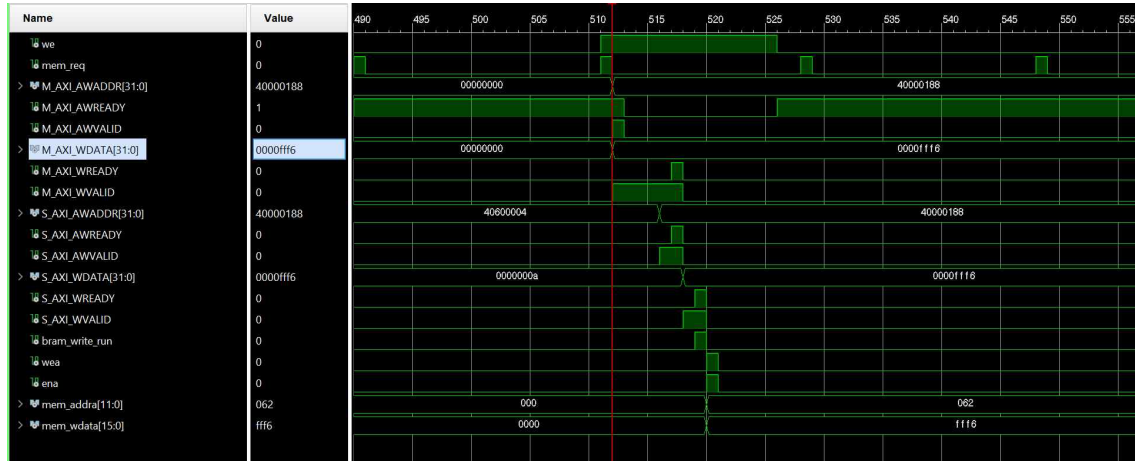


그림 5.8 ILA를 통한 AXI 쓰기 트랜잭션 및 BRAM 제어 신호 실측 파형

그림 5.8는 CPU가 메모리에 연산 결과를 기록하는 트랜잭션 파형이다. 마스터가 쓰기 주소를 인가하는 즉시 슬레이브의 AWREADY 신호가 맞물리며 단 1 클럭의 지연도 없는 (Zero Wait-State) 고속 핸드셰이크가 성사됨을 보여준다. 인가된 AXI 버스 주소는 BRAM 포트 주소로 정확히 변환되며, 목적 데이터가 제어 신호(wea, ena)와 함께 물리 메모리에 오차 없이 기록됨을 확인하였다.

(2) AXI 읽기(Read) 동작 실측

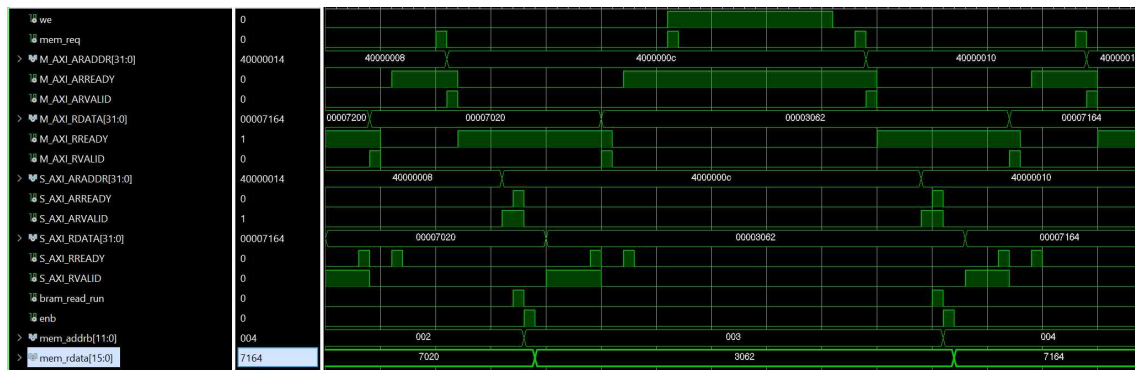


그림 5.9 ILA를 통한 연속 읽기 트랜잭션 및 BRAM 레이턴시 동기화 실측 파형

그림 5.9은 명령어를 연속으로 당겨오는 Fetch 과정을 캡처한 모습이다. 마스터가 4바이트 단위로 주소를 연속해서 달라고 요청하면, 슬레이브 컨트롤러는 BRAM 하드웨어 특유의 1 클럭 물리적 지연에 맞춰 영리하게 동기화 작업을 수행한다. 실제로 찍어보니 그 지연 시간에 딱 맞추어 유효 신호(S_AXI_RVALID)가 상승하고, 소중한 명령어 스트림이 1비트도 흘리지 않고 CPU 코어로 빨려 들어가는 걸 목격할 수 있었다. 이렇게 ILA로 건져 올린 두 파형을 대조해 보니 5.1절에서 돌려봤던 시뮬레이션 결과와 놀랍도록 일치했다. 결국 우리가 공들여 설계한 아키텍처가 실물 보드의 배선 지연을 이겨내고 타이밍 규격을 충족했으며, 하드웨어 레벨에서도 CPU와 메모리 간의 정합성을 완벽히 확보했음을 실증하는 대목이다.

5.3 외부 인터페이스를 통한 최종 시스템 통합 및 물리적 검증

시뮬레이션을 통한 내부 로직 분석(5.1절)과 ILA 실측 타이밍 검증(5.2절)이라는 관문을 통과했으나, 이제 사용자 입장에서 체감할 수 있는 최종적인 물리적 통합 검증을 진행했다. 복잡한 소프트웨어 시뮬레이터나 내부 디버깅 툴에 의존하지 않고, 실제 보드에 달린 7-Segment나 UART 같은 현실 장치들로 CPU의 결과물이 유실 없이 제대로 표출되는지 점검하여 전체 SoC 플랫폼의 무결성을 입증하고자 했다. 이 단계에서는 CPU 내부 누산기(AC)가 계산해 낸 최종 데이터(031E)가 AXI 버스를 내달려 각 이기종 슬레이브에 꽂힌 뒤, 불빛(LED)과 글자(터미널)라는 물리적 시각 정보로 변환되는 전 과정을 확인하는 데 주력했다.

5.3.1 7-Segment 디스플레이를 통한 시각적 하드웨어 구동 검증

설계된 CPU 코어가 연산한 16비트 이진수 결과값(Hex: 031E)은 AXI 버스를 통해 AXI Segment Slave 모듈의 레지스터로 인가된다. 수신된 데이터는 slave 내부의 하드웨어 Double Dabble 알고리즘 회로를 거쳐 10진수 숫자로 변환된 후 LED 핀 제어 로직을 통해 점등된다.

실제 FPGA 보드를 구동하여 물리적 검증을 수행한 결과, 그림 5-5와 같이 내부 AC 레지스터의 값인 0x031E에 해당하는 숫자가 7-Segment LED 디스플레이 상에 유실 없이 정확하게 시각적으로 점등됨을 확인하였다. 이는 CPU 코어, AXI 버스 인터커넥트, 그리고 최종 하드웨어 IP인 7-Segment가 물리적인 전기 신호 수준까지 동기화되어 하나의 통합된 시스템으로 성공적으로 구동됨을 의미한다.

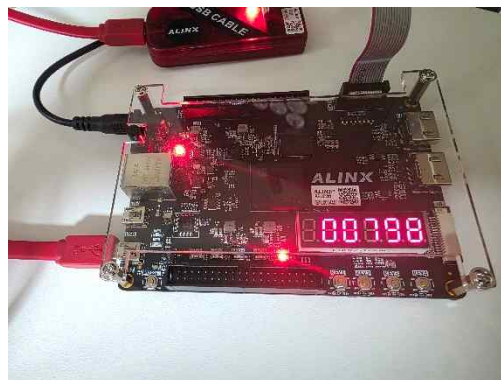


그림 5.10 FPGA 보드 기반 7-Segment 시각적 출력 최종 구동 결과

5.3.2 UART 직렬 통신을 통한 이종(Heterogeneous) 기기 간 통신 검증

단일 보드 내부의 구동을 넘어, 외부 PC와의 직렬 통신을 통해 이종 기기 간의 물리적 데이터 전송 능력을 교차 검증하였다. CPU가 UART 컨트롤러의 베이스 주소로 연산 데이터(031E)를 Write 하면, slave 내부의 하드웨어 ASCII 변환 모듈은 이 16비트 데이터를 '0', '3', '1', 'E'라는 4개의 ASCII 문자 코드로 변환한다.

최종적으로 FPGA 보드와 외부 PC를 UART 케이블로 결선하고 PC 측 터미널 프로그램(Tera Term)으로 모니터링한 결과, 그림 5-6과 같이 보드 상의 7-Segment에서 확인한 것과 동일한 값인 031E 문자열이 유실이나 깨짐 없이 PC 화면에 정확하게 줄바꿈되어 수신됨을 확인하였다.

결과적으로, 7-Segment와 UART라는 다른 물리적 특성을 가진 두 인터페이스에 동일한 데이터가 정확히 표출됨으로써, 본 연구에서 설계한 AXI 기반 SoC 플랫폼이 CPU의 내부 연산 로직부터 외부 다른 기기와의 물리적 통신 영역까지 오차 없이 완벽하게 동작함을 최종 증명 완료하였다.

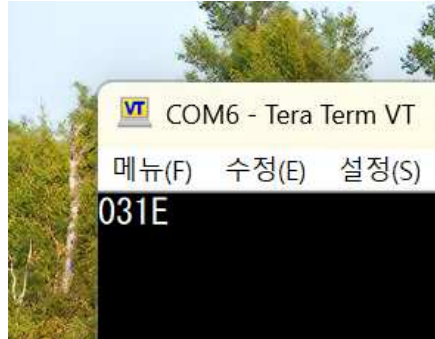


그림 5.11 외부 PC 터미널 프로그램을 통한 UART 직렬 통신 수신 최종 결과

제 VI 장 결론

이번 프로젝트에서는 단순히 시뮬레이션으로 동작만 확인하고 끝나는 기존 학부 수준의 CPU 설계에서 한 걸음 더 나아가, 산업 표준 규격인 AXI4-Lite 프로토콜을 올린 커스텀 CPU 시스템을 실제 FPGA 보드 위에 성공적으로 구현해 냈다. 원활한 시스템 통합을 목표로 기존의 공통 버스(Common Bus) 구조를 과감히 빼고 레지스터를 직접 연결(Point-to-Point)하는 방식을 택했으며, 버스의 응답 지연 상황에 맞춰 FSM 대기(Stall) 로직을 추가해 데이터 전송 과정의 병목을 크게 줄였다. 이에 더해 16비트로 동작하는 CPU와 32비트 AXI 버스 간의 규격 차이를 매우고자 전용 변환 모듈을 직접 짜고 BRAM 및 UART 등 주변장치를 위한 독자적인 슬레이브 컨트롤러까지 구축하면서, Memory-Mapped I/O 방식이 적용된 종합적인 SoC 아키텍처를 최종 완성할 수 있었다.

직접 만든 이 시스템이 논리적, 물리적으로 빈틈없이 돌아가는지 확인하고자 총 세 가지 단계로 꼼꼼하게 교차 검증을 진행했다. 가장 기본인 RTL 시뮬레이션 단계에서는 CPU 내부 파이프라인 흐름과 AXI 채널의 핸드셰이크 규격이 어긋남 없이 맞물리는지 파악했다. 이후 50MHz 실 클럭이 인가된 하드웨어 환경으로 넘어가 정적 타이밍 분석(STA)과 ILA 기능을 동원했고, 이를 통해 물리적인 배선 지연을 극복한 안정적인 타이밍 마진을 확보하면서 실제 칩 내부의 버스 트랜잭션까지 오차 없이 실측해 냈다. 마지막으로 CPU가 도출한 계산 결과가 7-Segment의 시각적 점등과 외부 PC의 UART 터미널 문자열로 유실 없이 온전히 표출되는 것을 두 눈으로 확인하며, 코어 내부부터 이중 기기와의 외부 통신까지 완벽하게 맞물려 동작함을 실증했다.

이론적인 아키텍처 설계를 넘어, 실제 산업 표준 버스를 붙여 실물 하드웨어 통합 검증까지 끝마쳤다는 데 꽤 큰 의미를 둔다. 얻은 AXI 제어 경험을 디딤돌 삼아, 앞으로는 지금의 단일 마스터 체제를 다중 마스터와 다중 슬레이브(Multi-Master, Multi-Slave)가 얹히는 복합적인 구조로 한 단계 키워보고자 한다. 나아가 복잡한 통신을 정리해 줄 버스 중재 로직(Arbiter)을 더하고 UART 외에도 더 다양한 주변장치를 연결하여, 지금보다 한층 고도화되고 실무에 가까운 SoC 플랫폼으로 발전시키는 것을 다음 연구 목표로 계획하고 있다.

참고 문헌

- [1] Vinay K, Ingle. John G. Proakis "Digital Signal Processing using Matlab"
1999 by prentice-Hall
- [2] Irael Koren, "Computer Arithmetic Algorithms", 1993 by Prentice-Hall
- [3] Neil H. E. Weste and Kamran Eshraghian "Principles of CMOS VLSI
Design" second edition by Addison Wesley
- [4] John G. Proakis, Dimitris G. Manolakis "Digital Signal Processing"

ABSTRACT

AXI 기반 CPU 설계 및 FPGA 구현

곽도영, 김기연, 남상우, 남승현

Dept. of Electronics Electrical Engineering

Dongguk University

SEOUL KOREA

본 연구는 시뮬레이션 위주의 기존 학부 수준 CPU 설계의 한계를 극복하고, 산업 표준인 AXI4-Lite 프로토콜을 적용한 16비트 커스텀 CPU를 실제 FPGA 플랫폼에 성공적으로 구현하는 것을 목표로 한다. 원활한 시스템 통합을 위해 기존 공통 버스(Common Bus) 방식을 레지스터 간 직접 연결(Point-to-Point) 방식으로 대체하고, 버스 응답 지연을 처리하기 위한 FSM 대기(Stall) 로직을 도입하여 아키텍처를 최적화하였다. 또한, BRAM, UART, 7-세그먼트 디스플레이를 위한 전용 변환 모듈 및 커스텀 슬레이브 컨트롤러를 개발하여 견고한 Memory-Mapped I/O 기반 SoC 아키텍처를 구축하였다.

설계된 시스템의 논리적, 물리적 무결성은 3단계 교차 검증 과정을 통해 엄격히 입증되었다. 첫째, RTL 시뮬레이션을 통해 명령어 파이프라인과 AXI 채널 핸드셰이크의 동기화를 확인하였다. 둘째, 50MHz 클럭 환경에서 정적 타이밍 분석(STA)과 ILA를 활용해 물리적 타이밍 마진과 칩 내부 버스 트랜잭션을 검증하였다. 마지막으로, 물리적 출력 검증을 통해 CPU의 연산 결과가 데이터 유실 없이 7-세그먼트 LED에 정확히 표시되고 UART를 통해 외부 PC 터미널로 전송됨을 입증하였다. 결론적으로 본 연구는 이론적인 프로세서 설계를 넘어, 표준 버스 기반의 실용적인 하드웨어 통합 검증을 성공적으로 완수했다는 데 의의가 있다.